

Robust Candidate MaxWeight Certificates for Heterogeneous Compute Scheduling: A Scheduleurm Case Study

(Authors' names are not included for peer review)

Authors are encouraged to submit new papers to INFORMS journals by means of a style file template, which includes the journal title. However, use of a template does not certify that the paper has been accepted for publication in the named journal. INFORMS journal templates are for the exclusive purpose of submitting to an INFORMS journal and are not intended to be a true representation of the article's final published form. Use of this template to distribute papers in print or online or to submit papers to another non-INFORM publication is prohibited.

Abstract. Heterogeneous compute clusters run workloads whose service rates depend on co-location, host pressure, accelerator memory, and topology. We model this as a configuration-action stochastic processing network and study a robust candidate MaxWeight policy that keeps the full action space in the capacity statement while charging candidate approximation, conservative service estimation, penalties, and approximate optimization against explicit slack. If the residual slack margin is positive, the policy satisfies a finite-set Foster recurrence certificate under bounded conditional second moments; the fixed-family and statewise variants are formalized in Lean. We instantiate this replay and oracle-audit pipeline in Scheduleurm; its live placement hook is not claimed to be a deployed global MaxWeight implementation. On measured finite slices, Scheduleurm improves all-job completion over legacy by 11.926-fold, 1.068-fold, 1.508-fold, and 1.160-fold on four workload buckets. Across 120 Poisson and bursty load-sweep replay scenarios, the adaptive candidate completes all jobs, improves legacy makespan and mean flow geometrically by 1.643-fold and 4.640-fold, and is not Pareto-dominated by state-of-the-art-inspired policy-semantics replay baselines on the same measured service cache. Claims are limited to audited measured slices and a controlled completed-active population, not raw history or direct external scheduler executions.

Key words: stochastic processing networks; MaxWeight scheduling; heterogeneous clusters; graphics-processing-unit co-location; robust optimization; formal verification; queueing networks

Subject classifications: Queues: scheduling; stochastic models: processing networks; computing: resource allocation

Area of review: Stochastic Models; Computing

1. Introduction

Schedulers for modern research clusters operate in a regime that is poorly described by a fixed resource vector. A reinforcement learning (RL) job can retain near-solo progress when several

copies share a graphics processing unit (GPU), whereas a dense GPU kernel can lose finite-batch delay performance when the same GPU is aggressively multiplexed. Central processing unit (CPU)-heavy evaluation jobs stress host cores and memory rather than accelerator compute, and short control-plane jobs expose queueing overhead and fragmentation. These cases are not exceptions to a simple model; they are the model. The scheduler chooses global configurations whose service rates depend on the set of jobs, their placement, co-location profile, and local hardware state.

The operations-research difficulty is that the natural action is a global configuration. Enumerating all placements and co-location patterns is combinatorial, but reducing the problem to a small list of hand-picked “sweet spots” can invalidate stability claims. A policy may look efficient on a finite batch while silently using an infeasible profile, or it may be throughput-optimal in a model that is disconnected from the measured service map. Our starting point is therefore to keep the full action family in the mathematical statement and to treat the implemented candidate family as an approximation whose loss must be paid from explicit slack.

The main theoretical contribution is a robust candidate MaxWeight theorem with complete slack accounting. The theorem states exactly how candidate approximation, conservative service estimation, scheduling penalties, and approximate optimization consume capacity slack. When the remaining margin is positive and the stochastic primitives have bounded conditional second moments, the resulting policy satisfies a finite-set Foster recurrence certificate. The result is stated for both fixed feasible families and state-dependent feasible families, the latter being the case relevant to live schedulers whose feasible actions depend on available jobs, alive nodes, and measured capacity boundaries.

The empirical contribution is not a claim that a single implementation has solved cluster scheduling in full generality. We instantiate the theorem in *Scheduleurm*, an existing scheduler used for local research workloads, and report bounded pieces of evidence. First, measured service curves close declared finite slices for four workload buckets: local light-control tasks, GPU-heavy JAX numerical-computing matrix-multiplication tasks, local CPU-heavy tasks, and Robust-Ensemble Soft Actor-Critic (RE-SAC) / Bayesian Amnesic Piecewise-Robust (BAPR)-like hybrid RL tasks. Second, explicit task-list replay compares the current policy with the legacy policy and with SOTA-inspired policy-semantics replay baselines, all using the same measured service cache, under static and online arrivals. Third, holdout diagnostics, finite lower-service capacity certificates, ablation, production coverage, and live-state dry-run trace audits show which theorem conditions are closed and which remain finite-slice claims. These results are useful because they connect the proof

Table 1 Claim matrix for scope, evidence, and non-claims.

Claim	Scope	Evidence	Not claimed
Stability theorem	Fixed and statewise candidate families satisfying the slack, lower-service, penalty, oracle-error, and moment assumptions.	Human proof plus Lean artifact for exact and approximate robust candidate MaxWeight.	Automatic stability of an arbitrary production scheduler row.
Finite-slice instantiation	Declared q00, q01, q10, and q11 measured action slices and the mixed portfolio built from those slices.	Measured service cache, capacity boundaries, explicit replay, slack-accounting certificate, and exact measured-slice fabric-cover calibration. A future-admitted measured-state gate certifies identity projection with $\rho = 0$ for any future task admitted to an exact measured service profile. A declared finite-domain positive-cover gate covers the exact measured service-cache universe, while a conservative all-state safety gate routes any unmeasured state to a zero-service probe/defer action.	Arbitrary unseen workloads, hardware regimes, or all-state fabric cover with positive lower service outside the admission contract.
Live scheduler hook	Optional one-task placement scorer and admission gates in Scheduleurm.	Code path, parameterized policy interface, and live-state dry-run trace audit, including a certified lower-service robust MaxWeight hook for service-admitted GPU candidates, plus a bounded launched ScheduleurmBench q01/q11 dispatch trace and a non-invasive active-production shadow trace over the same hook. The optional adaptive hook emits active-bucket sampler and regime-detector audit fields.	The default live scheduler has enabled the adaptive policy, or every future dispatch bypasses admission/probe before becoming theorem-facing.
Production bridge	Completed-active mapped Scheduleurm-controlled population.	Service-map lower-service oracle bridge, production coverage drilldown, and service-domain admission gate. A future-production admission contract routes service-certified active/queued jobs to theorem trace and unmeasured jobs to probe. A safe production launch/completion gate reports current large-scale active-production progress and refuses launches when the queue/resource conditions are unsafe. A controlled completion gate recognizes the existing bounded launched completion trace and defines the unmet 32-task and organic production thresholds.	Raw 30-day history, attempted-only jobs, or unmeasured future jobs are closed; or that large-scale launched production completion has been collected without queued jobs and idle resources.
External scheduler comparison	SOTA-inspired policy semantics replayed on the same measured service cache.	Online policy-semantics matrix for throughput, delay, and interference-control semantics, plus a direct-baseline scaffold that records endpoint smoke checks, a Gavel native generated-jobs smoke, bounded native Gavel simulator microbaseline rows for q01/q11 trace windows, same-workload adapter seeds, a Gavel service-unit equivalence certificate, and full-stack blockers.	Direct binary or full-stack superiority over Gavel, Pollux, Sia, IADeep, or Salus.
Learning/regime extensions	Active-bucket and hidden-regime certificates outside the main stability claim.	Deterministic active-bucket union bound, dwell/switching accounting, and a concrete deterministic round-robin sampler plus bounded two-window detector probability certificate.	The current live scheduler has enabled adaptive learning by default or has A/B performance evidence for it.

This table is the contract used to interpret all theorem, experiment, and artifact claims in the rest of the paper.

obligations to scheduler artifacts; they do not replace broader perturbation profiling for a global fabric-cover constant.

The paper is organized as follows. We first state the claim scope and reviewer-facing evidence contract. We then introduce the configuration-action processing network, give the candidate approximation and robust MaxWeight stability theorem, describe the Scheduleurm instantiation, report the computational evidence, position the work relative to scheduler and queueing literatures, and close with the remaining calibration limits.

2. Scope and Contributions

Table 1 states the reviewer-facing claim boundary used throughout the paper. The table is not a disclaimer appended after the fact; it is part of the methodology. Each computational claim is tied to the population and artifact that make it auditable, and each row also states the adjacent claim that is intentionally not made.

Table 2 Notation ledger.

Symbol	Meaning
$\mathcal{I}, i$	finite set of job classes and a class index
$t, Q_i(t), Q(t)$	slot, class- i backlog, and queue vector
a, a_t	configuration action and chosen action
$\mathcal{A}^{\text{full}}, \mathcal{A}^{\text{cand}}$	full action family and fixed candidate family
$\mathcal{A}_t^{\text{cand}}, \mathcal{A}^{\text{meas}}$	statewise candidate family and measured action slice
$Z_t, \omega_t, A_i(t), S_i(t)$	regime, exogenous randomness, class- i arrival, and class- i service
$\mu^z(a), \mu(a), \underline{\mu}_t(a)$	regime service mean, main-theorem service mean, and conservative lower-service estimate
$C(\mathcal{A}), \Lambda(\mathcal{A}), \Delta(\mathcal{A}), x_a$	service convex hull, downward-closed load region, probability simplex, and stationary mixing mass on action a
$H_{\mathcal{A}}(q), q$	support function and nonnegative support vector
$\Phi, d_{\Phi}, L, \rho, \pi$	finite action features, fabric metric, service Lipschitz envelope, cover radius, and candidate projection
$K_t(a), G_t(a), P_0, \beta$	switching cost, risk/interference penalty, additive penalty bound, and queue-scaled penalty coefficient
θ_{prof}	dimensionless profile-penalty fraction used by the queue-adaptive replay candidate
$\delta, \epsilon_{\text{cand}}, \epsilon_{\text{est}}, \alpha_0, \alpha_1, \eta$	full-region slack, candidate loss, estimation loss, additive oracle error, queue-scaled oracle error, and residual drift margin
B, N, V	second-moment bound, finite-set threshold, and quadratic Lyapunov function
$k, n, M_k(n), F_k(n)$	measured co-location profile, task-list size, all-job makespan proxy, and mean-flow proxy

The notation avoids reusing one symbol for multiple meanings. Acronyms are expanded at first textual use and then used consistently.

3. Model

3.1. Notation

Table 2 fixes the symbol ledger used throughout the paper. Each mathematical symbol is reserved for the meaning shown in the table; in particular, Q is the queue vector, q is an arbitrary nonnegative support weight, K_t is switching cost, and G_t is the risk or interference penalty.

3.2. Configuration actions

Let $\mathcal{I}$ be a finite set of job classes. Queue $Q_i(t) \in \mathbb{Z}_+$ counts the amount of unfinished work of class i at the beginning of slot t . A global configuration action $a \in \mathcal{A}^{\text{full}}$ specifies which jobs are served, which resource configuration each job receives, and where each job is placed. In a GPU cluster this includes, for example, the number of jobs sharing a GPU, CPU worker counts, memory pressure buckets, and host or device placement. The model intentionally treats these as one action rather than as independent per-job choices.

Given a regime z and exogenous randomness ω , action a generates a nonnegative service vector $S(a, z, \omega) \in \mathbb{R}_+^{|\mathcal{I}|}$. We write $\mu^z(a) = \mathbb{E}[S(a, z, \omega)]$. For the main theorem we first suppress z and use

$\mu(a)$; uniform-in-regime and average-regime extensions require additional switching assumptions and are not used as main empirical claims.

The queue dynamics are

$$Q_i(t+1) = [Q_i(t) - S_i(t)]^+ + A_i(t), \quad i \in \mathcal{I}, \quad (1)$$

where $A_i(t)$ is the class- i arrival in slot t . In the finite-support specialization used by the formal proof, the conditional distribution of $(A(t), S(t))$ depends on $Q(t)$ through a finite sample space. In the more general theorem it is enough to assume bounded conditional second moments.

3.3. Capacity and support functions

For a finite action family $\mathcal{A}$, define the service region

$$C(\mathcal{A}) = \left\{ \sum_{a \in \mathcal{A}} x_a \mu(a) : x \in \Delta(\mathcal{A}) \right\},$$

and its downward closure

$$\Lambda(\mathcal{A}) = \left\{ \lambda \in \mathbb{R}_+^{|\mathcal{I}|} : \exists v \in C(\mathcal{A}) \text{ with } \lambda \leq v \right\}.$$

The support function in the nonnegative orthant is

$$H_{\mathcal{A}}(q) = \max_{a \in \mathcal{A}} q^\top \mu(a), \quad q \in \mathbb{R}_+^{|\mathcal{I}|}.$$

Here $\Delta(\mathcal{A})$ is the probability simplex over the action family $\mathcal{A}$, and x_a is the stationary mixing mass assigned to action a . The vector q is a nonnegative support weight; it is not the queue state unless explicitly specialized to $Q(t)$. If λ has coordinate slack $\delta > 0$ in the full region, then there is a stationary mixture $x \in \Delta(\mathcal{A}^{\text{full}})$ such that $\lambda_i + \delta \leq \sum_a x_a \mu_i(a)$ for every i . Consequently,

$$q^\top \lambda + \delta \|q\|_1 \leq H_{\mathcal{A}^{\text{full}}}(q), \quad q \geq 0. \quad (2)$$

This elementary support implication is the bridge from capacity to MaxWeight drift; it is not an operational stability “if and only if” by itself.

3.4. Candidate families and fabric metrics

The implemented scheduler optimizes over $\mathcal{A}^{\text{cand}} \subseteq \mathcal{A}^{\text{full}}$. To avoid turning this computational restriction into an unaccounted model change, define a finite feature map $\Phi : \mathcal{A}^{\text{full}} \rightarrow \mathbb{R}^d$ and weighted ℓ_1 fabric metric

$$d_{\Phi}(a, a') = \sum_{r=1}^d w_r |\Phi_r(a) - \Phi_r(a')|.$$

The features may include co-location profile, node bucket, CPU bucket, memory pressure bucket, placement locality, and other scheduler-visible fabric attributes. We say that $\mathcal{A}^{\text{cand}}$ is a ρ -cover of $\mathcal{A}^{\text{full}}$ if for each full action a there is a candidate action $\pi(a) \in \mathcal{A}^{\text{cand}}$ with $d_{\Phi}(a, \pi(a)) \leq \rho$. If $\|\mu(a) - \mu(a')\|_{\infty} \leq L d_{\Phi}(a, a')$, then

$$H_{\mathcal{A}^{\text{full}}}(q) \leq H_{\mathcal{A}^{\text{cand}}}(q) + L\rho \|q\|_1, \quad q \geq 0. \quad (3)$$

This statement is used as a theorem assumption unless a perturbation-profiling table certifies Φ, L, ρ . For exact measured finite slices in which the candidate family enumerates the measured actions, we set $\rho = 0$.

4. Robust Candidate MaxWeight

4.1. Policy

Let $\underline{\mu}_t(a)$ be a conservative lower-service estimate for candidate action a at time t . Let $K_t(a) = K(a_{t-1}, a)$ model switching or rollback cost from the previous action to a , and let $G_t(a)$ model risk or interference penalties. The exact robust candidate MaxWeight rule is

$$a_t \in \arg \max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q(t)^{\top} \underline{\mu}_t(a) - K_t(a) - G_t(a) \right\}. \quad (4)$$

For practical schedulers, the selected action may be only approximately optimal. We assume the queue-scaled oracle condition relative to (4),

$$\begin{aligned} & \max_{a \in \mathcal{A}_t^{\text{cand}}} \left\{ Q^{\top} \underline{\mu}_t(a) - K_t(a) - G_t(a) \right\} \\ & \quad - \left\{ Q^{\top} \underline{\mu}_t(a_t) - K_t(a_t) - G_t(a_t) \right\} \\ & \leq \alpha_0 + \alpha_1 \|Q\|_1. \end{aligned} \quad (5)$$

Exact maximization is the special case $\alpha_0 = \alpha_1 = 0$.

4.2. Main stability certificate

THEOREM 1 (Robust candidate MaxWeight stability certificate). *Consider a finite system with job classes $\mathcal{I}$ and queue dynamics (1). Suppose:*

1. λ has full-action support slack δ , i.e., (2) holds.
2. The candidate family has support loss at most $\epsilon_{\text{cand}} \|q\|_1$, where $\epsilon_{\text{cand}} = L\rho$ under the fabric-cover condition (3).
3. The lower-service map has support error at most $\epsilon_{\text{est}} \|q\|_1$: $H_{\mathcal{A}^{\text{cand}}}(q) \leq \max_{a \in \mathcal{A}^{\text{cand}}} q^\top \underline{\mu}(a) + \epsilon_{\text{est}} \|q\|_1$.
4. Penalties satisfy $0 \leq K_t(a_t) + G_t(a_t) \leq P_0 + \beta \|Q(t)\|_1$.
5. The selected action satisfies the approximate oracle condition (5).
6. Conditional moments satisfy $\mathbb{E}[A_i(t) \mid Q(t) = Q] \leq \lambda_i$, $\underline{\mu}_i(a_t) \leq \mathbb{E}[S_i(t) \mid Q(t) = Q]$, and

$$\mathbb{E} \left[\frac{1}{2} \sum_i (A_i(t)^2 + S_i(t)^2) \mid Q(t) = Q \right] \leq B.$$

If

$$\eta = \delta - (\epsilon_{\text{cand}} + \epsilon_{\text{est}} + \beta + \alpha_1) > 0,$$

then for $V(Q) = \frac{1}{2} \sum_i Q_i^2$ and $\Delta V(t) = V(Q(t+1)) - V(Q(t))$,

$$\begin{aligned} \mathbb{E}[\Delta V(t) \mid Q(t) = Q] &\leq B + P_0 + \alpha_0 \\ &\quad - \eta \|Q\|_1. \end{aligned}$$

Consequently, for any N satisfying

$$B + P_0 + \alpha_0 + \gamma \leq \eta N$$

for some $\gamma > 0$, the process has a negative expected drift outside $\{Q : \|Q\|_1 \leq N\}$, and hence satisfies a finite-set Foster recurrence certificate.

The Lean 4 proof artifact (de Moura and Ullrich 2021) proves the fixed-family theorem and a statewise version in which the feasible family $\mathcal{A}_t^{\text{cand}}$ depends on the queue state, available jobs, and measured capacity-boundary rows. It also proves a bounded finite-support specialization in which B is constructed from coordinate sample bounds. In the artifact map, the paper-facing results are indexed as the calibrated approximate-oracle theorem and the statewise calibrated approximate-oracle theorem.

4.3. Proof decomposition

The proof has four parts. First, full-action slack implies the support bound (2). Second, the fabric cover transfers support from $\mathcal{A}^{\text{full}}$ to $\mathcal{A}^{\text{cand}}$ with loss $L\rho \|q\|_1$. Third, lower-service error, penalties, and approximate optimization are subtracted from the same slack budget. Fourth, the standard quadratic Lyapunov inequality

$$\begin{aligned} & V([Q - S]^+ + A) - V(Q) \\ & \leq \frac{1}{2} \sum_i (A_i^2 + S_i^2) \\ & \quad + Q^\top A - Q^\top S \end{aligned}$$

converts the deterministic support margin into conditional negative drift. The formalization keeps the operational necessity direction separate: an “operationally stabilizes” predicate is load-certified by a model that explicitly encodes the mean arrival vector, and zero-slack necessity follows from a conservation-law argument rather than from a definition.

5. Scheduleurm Instantiation

5.1. Measured action slices

Scheduleurm is an existing scheduler for local and remote research jobs. The new algorithmic layer deliberately separates two interfaces. The theorem-facing interface builds global or statewise candidate families with `queue_vector`, `lower_service`, `penalty_units`, and `score_semantics=robust_maxweight_lower_service`; service profiling, replay, policy-semantics baselines, slack accounting, and oracle audits live under `simulation/` and `algorithm/experiments/`. The deployed live interface remains an optional one-task placement scorer and admission-gate hook selected by policy name. It is useful for carrying calibrated candidates back into the scheduler, but it is not used in this paper as evidence that the production dispatcher already implements the global robust MaxWeight policy.

Table 3 summarizes the current measured finite slices. A capacity boundary is a measured or live sanity profile that is not usable as a robust feasible action. Once profile k is a boundary, profile k and all higher profiles are excluded from the theorem-facing candidate family for that workload bucket.

The replay policy uses a queue-adaptive robust MaxWeight penalty selector rather than a fixed co-location profile. For a workload with backlog count b , it first restricts profiles to a measured support envelope and then chooses profile k by the score

$$b \widehat{\mu}_i(k) - \theta_{\text{prof}} \widehat{\mu}_i^{\max} k, \quad \theta_{\text{prof}} = 0.10,$$

Table 3 Current measured finite slices.

Bucket	Feasible profiles	Boundary	Candidate	Legacy
q00 light_control_local	1–13	14	13	1
q01 gpu_heavy_jax_matmul	1–8	none	1 / 8	3
q10 cpu_heavy_local_bench	1–9	10	8	9
q11 hybrid_rl_resac_ant	1–9	10	2 / 3	5

q01 uses profile 1 in low-backlog online replay and profile 8 in the high-backlog support certificate. q11 uses profile 2 in online replay and profile 3 in the high-backlog support certificate and mixed portfolio. The q11 profile-10 replay point from earlier measurements is historical only: fresh live sanity showed runtime out-of-memory (OOM) at profile 10, making it the first robust capacity boundary for the current node bucket.

Table 4 Theorem-policy crosswalk in the Scheduleurm artifact.

Theorem object	Artifact field or module	Reviewer-facing interpretation
Queue vector Q	<code>queue_vector</code> in oracle traces and replay slots	Defines the backlog weights in $Q^\top \underline{\mu}(a)$.
Candidate action $a \in \mathcal{A}_t^{\text{cand}}$	<code>action_model.py</code> , <code>adaptive_trace_policy.py</code> , and per-slot candidate rows	Represents a global or statewide placement/co-location configuration for the theorem-facing oracle.
Lower service $\underline{\mu}(a)$	<code>service_model.py</code> and measured service cache	Conservative service row used in the drift inequality.
Penalty $K_t(a) + G_t(a)$	<code>penalty_units</code> , <code>penalty_fit.py</code> , and bounded tie-break terms	Switching, risk, and delay-control terms that consume bounded or queue-scaled slack.
Oracle error (α_0, α_1)	<code>oracle_audit.py</code> and support-gap reports	Measures approximate maximization loss relative to the theorem objective.
Live placement hook	<code>placement.py</code> and <code>skill/scheduler.py</code> policy selector	Optional scheduler integration point; theorem use requires trace enrichment and oracle audit.
OR closure runner	<code>or_submission_closure.py</code>	Runs online arrivals, holdout lower-service certificates, ablations, live-state trace audit, and Lean supplement repackaging without modifying <code>skill/scheduler.py</code> .

The theorem-facing rows expose robust MaxWeight score semantics explicitly; the live hook is an integration surface and not, by itself, a stability certificate.

where $\widehat{\mu}_i(k)$ is the measured aggregate service for class i at profile k , and $\widehat{\mu}_i^{\max}$ is the best measured aggregate service in the same envelope. Thus small queues may use lower-interference profiles such as q01 profile 1, while large queues move to support actions such as q01 profile 8. This implementation matches the approximate-oracle theorem: the support envelope preserves the drift action at large Q , and the finite-batch delay preference is a bounded profile penalty that is paid from the same slack budget as $K_t + G_t$.

Table 4 gives the implementation crosswalk. The purpose is to make the theorem-policy alignment auditable without pretending that every live dispatch call has already been upgraded to a global batch optimizer. When a row is generated by the theorem-facing pipeline, the robust MaxWeight objective is explicit. When a row is generated by the live hook, the row is an engineering action that must be enriched and audited before it is used as theorem evidence.

5.2. Auditable theorem objects

The Scheduleurm artifact distinguishes three populations. The explicit task-list replay population is used for performance comparisons. The completed-active production population is used for a service-map theorem oracle bridge. The raw scheduler history is treated only as telemetry: it contains cancelled, attempted-only, benchmark, and externally adopted jobs and is not a clean arrival stream for the theorem.

The current completed-active Scheduleurm-controlled production population has 3437 mapped records out of 3437, positive mapped-slice capacity slack, and a service-map robust MaxWeight lower-service oracle bridge with $\alpha_0 = \alpha_1 = 0$. A separate live-state dry-run trace audit closes 128 synthetic placement slots and 767 candidate rows under the `sweetspot_v1` hook against the current probed node state. The dry-run probe does not write the queue and does not launch tasks. It proves the trace/enrichment/audit pipeline is executable on current scheduler candidate families at a longer candidate-slot scale; it is still a live-state dry-run audit, not statistical evidence for all online dispatches. A bounded launched q01/q11 ScheduleurmBench trace separately exercises the robust lower-service hook on real tasks and passes its completion bridge. The 2026-06-12 production queued trace then closes the current queued production population: one admissible BAPR task, one theorem slot, and an oracle audit pass, with no queue mutation and no launch/completion claim. A non-invasive active-production shadow trace copies current running production records into shadow queued records, evaluates the same optional theorem hook without mutating the queue, and closes the GPU theorem subset with $\alpha_0 = \alpha_1 = 0$. This shadow trace is useful evidence for live candidate semantics, but it is not a launched production dispatch trace. The raw 30-day telemetry window has 6957 records, of which 4825 are mapped and 2132 are unmapped, so its global coverage flag is intentionally false and it is not used as the theorem population.

6. Computational Evidence

6.1. Explicit task-list replay

The main benchmark uses one explicit task list per quadrant and one mixed portfolio. All policies receive the same jobs, total work units, resource-pool counts, arrival times, and measured service cache. Static arrivals are used for the first comparison because all-job completion time is then unambiguous. The online-arrival gate then runs Poisson and bursty traces at load factors 0.50, 0.70, 0.85, and 0.95 with three random seeds for each task set.

Table 5 Candidate policy versus legacy Scheduleurms on explicit task lists.

Task list	Candidate profile	Makespan	Mean flow	p90 flow
q00 light control	light=13	11.926×	11.763×	11.851×
q01 GPU-bound compute	gpu=1	1.068×	1.114×	1.155×
q10 CPU-host bound	cpu=8	1.508×	1.534×	1.522×
q11 CPU/GPU coupled	hybrid=3	1.160×	1.174×	1.171×
Mixed portfolio	cpu=8, gpu=1, hybrid=3	1.158×	1.269×	1.153×

Ratios are legacy divided by candidate, so larger is better for Scheduleurm candidate. Each row uses the current robust feasible family; q11 profile 10 and above are excluded. p90 is the 90th percentile of job flow time.

Table 5 compares the current candidate policy with the legacy Scheduleurm caps. The metric is completion of all jobs in the task list, not the time to finish a single job.

The online-arrival closure gate contains 120 scenarios: five task sets, two arrival modes, four load factors, and three seeds. The adaptive candidate completed every job in every scenario. Relative to legacy caps, its geometric mean improvement was 1.643×

 for makespan and 4.640× for mean flow. The worst scenario-level mean-flow improvement over legacy was 1.031×; one q10 bursty makespan ratio was 0.9997×, within the 0.5% dominance tolerance used for sampled replay comparisons. The maximum observed candidate backlog was 114 jobs, and the maximum time-weighted mean backlog was 47.945 jobs.

The ablation gate separates the legacy caps, the scalar `sweetspot_v1` hook semantics, support-only makespan scoring, delay-only mean-flow scoring, statewise interference guarding, removal of the bounded profile penalty, and reversal of the profile tie-break. Across the 15 static, Poisson, and bursty traces, no ablated policy Pareto-dominated the full queue-adaptive robust lower-service scorer; the full candidate retained geometric improvements of 1.796×

 in makespan and 3.465× in mean flow over legacy.

6.2. Policy-semantics replay baselines

No single external scheduler covers all four Scheduleurm quadrants and the mixed portfolio without changing the workload model. We therefore compare to SOTA-inspired policy-semantics replay baselines on the same measured service cache: a throughput-table goodput policy representing Gavel, Pollux, and Sia-like schedulers; a finite-batch delay oracle representing shortest remaining processing time (SRPT), Gittins, and shortest expected remaining processing time (SERPT)-style policies (Scully et al. 2020); and an interference guard representing IADeep/Salus-like co-location control. This is a policy-semantics comparison, not a direct binary execution of those systems, and we do not use it to claim full-stack superiority over those schedulers. A direct-adaptor scaffold

Table 6 Online policy-semantics replay aggregate on identical measured service cache.

Policy-semantics baseline	Candidate / makespan	Candidate / mean flow	Candidate / p90 flow
Throughput table goodput	0.9995×	1.0845×	1.0791×
Delay oracle	1.0001×	0.9997×	1.0001×
Interference guard	1.0002×	1.0575×	1.0535×
Quadrant composite	1.0002×	1.0575×	1.0535×

Entries are geometric mean candidate-versus-policy ratios across the online arrival scenarios. Values above one favor the candidate. A row would dominate the candidate only if both makespan and mean-flow ratios were materially below one; no such row appears under the 0.5% replay tolerance.

also inspects the cloned external repositories: Pollux/AdaptDL and Decima entryptoints smoke-check locally, and an isolated Gavel copy passes dependency imports, protobuf stub generation, entryptoint help, a native generated-jobs simulator smoke, a native Gavel trace smoke, and a Scheduleurm q01 native trace-seed smoke. A stronger native Gavel microbaseline now runs bounded Scheduleurm-exported q01 and q11 trace windows through Gavel's own trace simulator: the q01 window completes 4/4 jobs with average job completion time 44.561 seconds and makespan 50.929 seconds, and the q11 window completes 8/8 jobs with average job completion time 20.900 seconds and makespan 22.297 seconds. These rows validate native simulator completion and metric extraction on bounded same-trace windows. A separate service-unit certificate now checks the Scheduleurm-to-Gavel trace adapter: q01 has 48 native trace rows, q11 has 160 native trace rows, both preserve total units and resource counts, and the throughput seed is present. Exact arrival times differ because the native Gavel trace uses a small deterministic ordering jitter, and no calibration experiment yet proves that a Gavel simulator step is the same service unit as a Scheduleurm measured lower-service unit. Gavel is therefore still not a full-stack same-workload performance baseline. Scheduleurm now emits same-workload seed artifacts for Gavel throughput/trace replay, AdaptDLJob manifests, IADeep pod manifests, Salus benchmark metadata, and Decima DAG metadata, but the direct runs remain blocked by native trace/schema/service-unit validation, Kubernetes, Docker, NVIDIA runtime, or scope mismatch. We report those smoke checks and adapter seeds as artifact readiness, not as comparative performance.

Table 6 reports the online policy-semantics aggregate over the 120 Poisson and bursty load-sweep scenarios. The candidate is not Pareto-dominated by any SOTA-style policy under the same measured service cache. The matrix also shows the nature of the tradeoff: the throughput-table endpoint is marginally faster on makespan, while the adaptive candidate improves flow metrics; the delay oracle is essentially tied on makespan and mean flow under this measured service cache.

On the q01 GPU-heavy list, the adaptive candidate chooses profile 1 in low-backlog online scenarios and a support action in high-backlog certificates. This resolves the earlier fixed-profile

tradeoff: the candidate keeps the delay-oriented behavior when the queue is small and moves to service-support actions when the queue is large. This queue dependence is the intended Pareto geometry of the robust MaxWeight penalty, not a hidden post-hoc rule.

6.3. Theorem-condition and live-sanity checks

The measured portfolio finite slice also instantiates the slack inequality in Theorem 1. With load $\lambda_i = 0.8 \mu_i(a_{\text{selected}})$, the candidate family equals the measured action slice, so $L\rho = 0$. The resulting certificate is

$$\begin{aligned}\delta &= 0.066921842, \\ \epsilon_{\text{est}} &= \beta = \alpha_1 = 0, \\ \eta &= 0.066921842.\end{aligned}$$

The finite-support second-moment bound is $B = 6361.354514$, and the resulting finite-set threshold is $N = 95072$. This is a theorem-condition certificate for a declared finite-support load model; it is not a claim that the static replay trace is itself a stationary arrival process. The zero modeled losses arise because this certificate uses an exact enumerated measured slice and an exact service-map oracle. They should not be read as evidence that a broad fabric-cover metric has already been calibrated with $L\rho = 0$, or that holdout lower-service error vanishes outside the measured rows. For smaller candidate families, the artifact now reports a greedy finite-feature k-center cover curve; those rows quantify the nonzero $L\rho$ that must be paid before invoking the fabric-cover theorem away from the exact measured slice.

The OR closure runner also computes a holdout lower-service diagnostic. Using a sparse empirical-Bernstein lower confidence bound against the original mean-service certificate is intentionally conservative and does not close the same slack: the selected-action diagnostic has $\epsilon_{\text{est}} = 43.1457$, giving negative adjusted η . We therefore do not claim high-confidence stochastic generalization from these sparse samples. Instead, the theorem-facing empirical certificate uses the finite measured lower-service model directly: the lower service is the observed finite-slice lower service, the declared load is 0.8 times that lower service, and $\epsilon_{\text{est}} = 0$ for that model by construction. Table 7 reports the resulting positive drift margins.

The replay model was checked against fresh progress-window live measurements on representative measured service-cache rows, not only on the final selected profiles. For the q01 profile-4 row, replay predicted a makespan of 1661.313 seconds, whereas the fresh live proxy was 1652.248 seconds, a relative error of 0.005457. For q11 profile 2, replay predicted 38382.365 seconds and

Table 7 Finite measured lower-service capacity certificates.

Task set	High-backlog selected profiles	η
q00 light control	light=13	123.0121
q01 GPU-bound compute	gpu=8	11.9469
q10 CPU-host bound	cpu=8	10.2331
q11 CPU/GPU coupled	hybrid=3	0.065934
Mixed portfolio	cpu=8, gpu=4, hybrid=3	0.065934

Each row uses the finite measured lower-service model with load fraction 0.8. It is a theorem-condition certificate for the declared finite slice, not a claim of stochastic generalization beyond the measured profiles.

the fresh live proxy was 37760.000 seconds, a relative error of 0.016215. For the mixed portfolio, composed live slices gave relative errors of 0.007349 on makespan, 0.005417 on mean flow, and 0.007964 on p90 flow. These are progress-window sanity checks; the long jobs were not all run to natural completion.

A separate launched live theorem-dispatch gate ran controlled q01/q11 ScheduleurmBench tasks through the optional robust lower-service hook. The bounded run launched 6 real tasks, emitted 7 theorem-grade dispatch slots and 13 candidate rows, and passed the completion bridge; one remote launch failed because the working directory was unavailable and was retried on a certified local candidate. This is a bounded live-dispatch sanity check, not a production-wide trace. An earlier 2026-06-12 production-queue snapshot reported PRODUCTION_WIDE_LIVE_TRACE_PASS for one queued production task. It is a read-only theorem trace, not a launched completion trace. The artifact also runs a shadow production probe. That probe reads the current queue and node state, copies active production records into shadow queued records, evaluates the optional `theorem_maxweight_v1` hook without queue mutation or task launch, and emits a nonempty robust lower-service theorem subset. The production launch/completion gate is rerunnable on rolling queue snapshots; recent snapshots found queued or active production work but high local GPU utilization, so the gate waited rather than launching unsafely. It still records active-production progress observations and shadow theorem slots. The theorem subset bridge is usable with $\alpha_0 = \alpha_1 = 0$; CPU fallback slots use legacy scheduler sort-key semantics and are deliberately excluded from the theorem subset.

6.4. Formal and production artifacts

The Lean proof artifact is submitted separately as a consolidated file `ScheduleurmUpload.lean`.

The checked artifact has Secure Hash Algorithm 256-bit (SHA-256) hash

af79be4416e4c4add0fe41663fc0927af7058fe04412908b6688a8409227f01b.

The commands `lake build Scheduleurm` and `lake env lean ScheduleurmUpload.lean` both pass. The current reviewer supplement rerun copied `ScheduleurmUpload.lean`, `lakefile.toml`, `lean-toolchain`, the paper-theorem to Lean-theorem crosswalk, and a path-independent build log. A comment-aware search for `sorry`, `admit`, and `axiom` returns no proof-term matches. The theorem names used in this paper are searchable in the upload file; the hash above is meaningful only together with the supplement manifest and build log.

For production data, the current theorem-facing population is the completed-active `Scheduleurm`-controlled set. It has 3437 strict mapped records out of 3437, with mapped-slice slack 9.123×10^{-6} . The service-map oracle bridge passes with $\alpha_0 = \alpha_1 = 0$. By contrast, the raw 30-day history has 6957 records, with 4825 mapped records and 2132 unmapped records, so its mapped fraction is about 0.694 and its global theorem-coverage flag is false. This distinction is important: the artifact proves coverage of a controlled completed-active population, not closure of every raw, attempted-only, external, or future scheduler row.

7. Related Work

7.1. Deep-learning and heterogeneous cluster schedulers

Goodput-oriented deep-learning schedulers such as Gavel, Pollux, and Sia use measured or modeled throughput tables to allocate heterogeneous accelerators (Narayanan et al. 2020, Qiao et al. 2021, Subramanya et al. 2023). Fairness-oriented systems such as Themis study efficient sharing under fairness constraints (Mahajan et al. 2020). GPU sharing systems such as Gandiva, Salus, and IADeep expose or exploit fine-grained co-location and interference behavior (Xiao et al. 2018, Yu and Chowdhury 2020, Chen et al. 2023). These systems motivate our measured-service replay baselines, but our theoretical question is different: we ask how a scheduler can restrict a combinatorial configuration action space to a candidate family while preserving a queueing-stability certificate with explicit loss terms.

Hybrid CPU/GPU placement and cluster allocation systems such as AlloX and related heterogeneous placement work also emphasize that accelerator scheduling cannot be reduced to GPU count alone (Le et al. 2020, Carvalho et al. 2024). `Scheduleurm` follows the same empirical lesson. Its `q00`, `q10`, and `q11` buckets are declared local or node-bucket service certificates, not claims that all CPU-heavy or RL workloads have the same service curve.

7.2. Queueing-network control and learning

MaxWeight and backpressure policies are classical tools for stabilizing constrained queueing systems under capacity-region slack (Tassiulas and Ephremides 1992, Stolyar 2004, Busic and Meyn 2015). Our proof follows this lineage but adds three implementation-facing terms: candidate-set approximation, lower-service estimation, and approximate-oracle loss. Work on state-dependent processor sharing and workload-dependent admission control studies service rates that depend on the number or composition of active jobs (Gupta and Zhang 2022, Bekker and Borst 2006, Altman et al. 2006). The Scheduleurm co-location curves are a systems instance of the same phenomenon.

Learning-while-scheduling work, including MaxWeight with discounted upper confidence bound (UCB) and learning-based queueing controls, studies unknown service statistics under online exploration (Yang et al. 2023, Liu et al. 2022, Dai and Gluzman 2022). We do not claim a complete online learning theorem for the current Scheduleurm sampler. The formal artifact proves conditional high-probability lifting for active-bucket certificates. The current experiment artifact closes the deterministic finite active-bucket union bound and replay dwell/switching accounting, and adds a concrete deployable probability model: deterministic round-robin forced exploration over the finite active bucket set and a bounded two-window mean-shift detector. The certificate covers 64 active buckets and 120 replay scenarios with minimum forced samples 1561 per bucket, Hoeffding radius 0.0523, detector union tail bound 0.00564, and detection-delay bound 1024 decisions. A 2026-06-12 optional live-state probe further verifies that `adaptive_theorem_maxweight_v1` emits active-bucket sampler and regime-detector audit fields while preserving the lower-service theorem oracle. This remains an opt-in extension path; the default live scheduler policy is not changed.

8. Discussion and Limitations

The main contribution of this work is a bridge between Operations Research (OR)-style queueing certificates and a concrete scheduler artifact. The theory does not shrink the full action space into the implemented candidate set. Instead, it states the loss incurred by that restriction and pays it from capacity slack. The Scheduleurm implementation then makes the proof objects auditable: measured lower-service rows, capacity boundaries, explicit task lists, finite-slice slack accounting, and lower-service oracle traces are all materialized as artifacts.

Several limitations remain. First, the strongest empirical claim is an exact measured finite-slice claim. We now calibrate the finite-feature metric and the service-sensitivity envelope on those measured slices, and the main measured certificate uses the exact candidate family with $\rho = 0$. The

greedy k-center cover curve reports how $L\rho$ changes as the candidate family is smaller than the measured full slice, but a broader positive-service future/all-state claim would still require perturbation evidence for the claimed state population, projection π , cover radius ρ , service envelope L , and failure probability if the envelope is statistical; the current artifact includes a metric-contract table for the measured finite population, including the feature map, numeric scales, projection population, exact ρ , compressed-cover $L\rho$, and exclusions. The future-admitted fabric-cover gate closes a restricted future claim by construction: if a future task is admitted to one of 113 exact positive measured workload domains, the theorem-facing projection is identity and $\rho = 0$; otherwise the task is probe-required and excluded from the positive-load theorem-facing stream. A declared finite-domain positive-cover gate formalizes that population as 193 exact service-cache buckets: 187 have positive lower service, six are measured capacity boundaries, and zero are uncovered. A separate conservative all-state safety gate covers every scheduler-visible state by partitioning it into measured-admitted states or an unknown probe/defer action with zero theorem service. This closes the declared finite positive domain and all-state safety, not arbitrary positive-service all-state stability. Second, the SOTA-inspired comparisons are replayed policy semantics on the same measured Scheduleurm service cache. The artifact includes a direct external-baseline scaffold for Gavel, Pollux/AdaptDL, IADeep, Salus, and Decima, and now emits same-workload adapter seeds for each of them. Gavel additionally has bounded native simulator q01/q11 microbaseline rows, and the Gavel service-unit certificate closes trace/schema compatibility while keeping `gavel_service_unit_equivalence_ready=false`. The strict full-stack superiority gate still rejects direct superiority claims: the current machine has no Docker, no Kubernetes command-line client, and no Go toolchain, and Gavel still lacks measured service-unit equivalence to Scheduleurm service rows. Pollux/AdaptDL and Decima have local entryptpoint smoke checks, but these smoke checks are not full-stack performance baselines. Third, the live oracle trace evidence separates dry-run candidate-family audits, bounded launched ScheduleurmBench dispatch, read-only current queued-production trace, and non-invasive active-production shadow probing. The bounded launched trace validates the hook on controlled tasks, the queued-production trace validates the current queued population without launch, and the shadow trace validates current active-production GPU theorem subset semantics without queue mutation. A future-production admission contract now routes active or queued production jobs through `ADMIT_THEOREM_TRACE` or `PROBE_REQUIRED`. The production launch/completion gate then enforces the operational side: the 2026-06-12 gate snapshots observed dozens of active production jobs, 24 progress observations,

and local GPU-utilization readings above the launch threshold, so they returned WAIT-style statuses and did not launch new work. The controlled production completion gate recognizes the existing bounded launched trace (six submitted controlled tasks, seven theorem slots, 13 candidate rows, and $\alpha_0 = \alpha_1 = 0$) but keeps the stronger 32-task controlled threshold and large-scale organic production completion flags false. This gives bounded controlled completion and large-scale active-production progress evidence, not a large-scale organic launched production completion trace. Fourth, q00 and q10 now have a broad measured-envelope gate over the service cache: one q00 light-control workload and 102 q10 CPU-like workloads have positive measured lower-service rows. This supports a measured-envelope claim for admitted q00/q10 classes, but not a universal claim over every CPU-heavy or data-loader-heavy program. The q11 result remains a declared robust node-bucket certificate and should not be generalized to all RL/BAPR deployments without replication on those buckets.

These limitations are not incidental. They are the boundary that makes the claims falsifiable and therefore stronger. The next empirical step is to run larger launched production-wide theorem traces when queued production jobs and low-interference resources are simultaneously available and, if desired, to provision the external Kubernetes/Docker/Go/runtime stacks needed for direct full-stack SOTA comparisons. The next theoretical-to-systems step is to A/B validate the optional active-bucket sampler and bounded two-window detector before promoting them from extension certificates to the default scheduler policy.

9. Conclusion

We formulated heterogeneous compute scheduling as a configuration-action stochastic processing network and proved a robust candidate MaxWeight stability certificate with explicit slack accounting. The proof identifies precisely how candidate approximation, lower-service estimation, penalties, and approximate optimization consume capacity slack. In Scheduleum, measured finite service slices, explicit task-list replay, online arrival stress replay, finite lower-service capacity certificates, production coverage, and live-state dry-run oracle-trace audits instantiate the theorem objects under a bounded claim scope. The current evidence supports a strong but limited conclusion: on declared measured slices, Scheduleum's robust adaptive candidate policy improves over its legacy caps and is not Pareto-dominated by the policy-semantics replay baselines, while the formal artifact verifies the stability spine without unproved Lean placeholders.

Code and Data Availability

The code, experiment artifacts, measured service-cache summaries, reviewer supplement manifest, and README files for this submission are provided in the supplementary Scheduleurm artifact package. The package contains the Scheduleurm source tree, the `algorithm/` and `simulation/` replay modules, the generated gate reports under `md/`, and the Lean supplement headed by `ScheduleurmUpload.lean`. Raw scheduler history records that are not part of the theorem-facing completed-active population are retained as operational telemetry and are not used as the claimed replication population.

References

- Altman E, Avrachenkov K, Ayesta U (2006) A survey on discriminatory processor sharing. *Queueing Systems* 53(1–2):53–63, URL <http://dx.doi.org/10.1007/s11134-006-7586-8>.
- Bekker R, Borst SC (2006) Optimal admission control in queues with workload-dependent service rates. *Probability in the Engineering and Informational Sciences* 20(4):543–570, URL <http://dx.doi.org/10.1017/S0269964806060335>.
- Basic A, Meyn S (2015) Stability properties of constrained queueing systems and their fluid models. *Queueing Systems* 81(2):87–143, URL <http://dx.doi.org/10.1007/s11134-015-9446-5>.
- Carvalho MNL, Simitsis A, Queralt A, Romero O (2024) Workload placement on heterogeneous CPU-GPU systems. *Proceedings of the VLDB Endowment* 17(12):4241–4244, URL <http://dx.doi.org/10.14778/3685800.3685845>.
- Chen W, Mo Z, Xu H, Ye K, Xu C (2023) Interference-aware multiplexing for deep learning in GPU clusters: A middleware approach. *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (ACM), URL <http://dx.doi.org/10.1145/3581784.3607060>.
- Dai JG, Gluzman M (2022) Queueing network controls via deep reinforcement learning. *Stochastic Systems* 12(1):30–67, URL <http://dx.doi.org/10.1287/stsy.2021.0081>.
- de Moura L, Ullrich S (2021) The Lean 4 theorem prover and programming language. URL <https://arxiv.org/abs/2107.00608>.
- Gupta V, Zhang J (2022) Approximations and optimal control for state-dependent limited processor sharing queues. *Stochastic Systems* 12(2):205–225, URL <http://dx.doi.org/10.1287/stsy.2021.0087>.
- Le TN, Sun X, Chowdhury M, Liu Z (2020) AlloX: Compute allocation in hybrid clusters. *Proceedings of the Fifteenth European Conference on Computer Systems*, 1–16 (ACM), URL <http://dx.doi.org/10.1145/3342195.3387547>.
- Liu B, Xie Q, Modiano E (2022) RL-QN: A reinforcement learning framework for optimal control of queueing systems. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 7(1):1–35, URL <http://dx.doi.org/10.1145/3529375>.

- Mahajan K, Balasubramanian A, Singhvi A, Venkataraman S, Akella A, Phanishayee A, Chawla S (2020) Themis: Fair and efficient GPU cluster scheduling. *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation*, 289–304 (USENIX Association), URL <https://www.usenix.org/conference/nsdi20/presentation/mahajan>.
- Narayanan D, Santhanam K, Kazhamiaka F, Phanishayee A, Zaharia M (2020) Heterogeneity-aware cluster scheduling policies for deep learning workloads. *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*, 481–498 (USENIX Association), URL <https://www.usenix.org/conference/osdi20/presentation/narayanan-deepak>.
- Qiao A, Choe SK, Subramanya SJ, Neiswanger W, Ho Q, Zhang H, Ganger GR, Xing EP (2021) Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 1–18 (USENIX Association), URL <https://www.usenix.org/conference/osdi21/presentation/qiao>.
- Scully Z, Harchol-Balter M, Scheller-Wolf A (2020) Simple near-optimal scheduling for the M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4(1):1–29, URL <http://dx.doi.org/10.1145/3379477>.
- Stolyar AL (2004) MaxWeight scheduling in a generalized switch: State space collapse and workload minimization in heavy traffic. *The Annals of Applied Probability* 14(1):1–53, URL <http://dx.doi.org/10.1214/aoap/1075828046>.
- Subramanya SJ, Arfeen D, Lin S, Qiao A, Jia Z, Ganger GR (2023) Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 642–657 (ACM), URL <http://dx.doi.org/10.1145/3600006.3613175>.
- Tassiulas L, Ephremides A (1992) Stability properties of constrained queueing systems and scheduling policies for maximum throughput in multihop radio networks. *IEEE Transactions on Automatic Control* 37(12):1936–1948, URL <http://dx.doi.org/10.1109/9.182479>.
- Xiao W, Bhardwaj R, Ramjee R, Sivathanu M, Kwatra N, Han Z, Patel P, Peng X, Zhao H, Zhang Q, Yang F, Zhou L (2018) Gandiva: Introspective cluster scheduling for deep learning. *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation*, 595–610 (USENIX Association), URL <https://www.usenix.org/conference/osdi18/presentation/xiao>.
- Yang Z, Srikant R, Ying L (2023) Learning while scheduling in multi-server systems with unknown statistics: MaxWeight with discounted UCB. *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 4275–4312 (PMLR), URL <https://proceedings.mlr.press/v206/yang23d.html>.
- Yu P, Chowdhury M (2020) Salus: Fine-grained GPU sharing primitives for deep learning applications. *Proceedings of Machine Learning and Systems*, volume 2, 600–615, URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/d9cd83bc91b8c36a0c7c0fcca59228f2-Abstract.html.